Vanderbilt University Motorsports
Vanderbilt University | VUM-27 | 2027 Season

HOW-TO GUIDE

GENERAL / ONBOARDING

GitHub for New Members

The first push, then the CAD repository and the working cycle

GEN-HG-001 • Revision **D** • 2026-08-17 • Status: **Released**

Summary

How the team uses GitHub, in two stages. The first stage, and the second half of onboarding, is pushing your member introduction to a repository where nothing else depends on the contents, and this exercise covers the whole workflow. The second stage applies the same steps to the VUM-27 CAD repository. Most of the team uses GitHub Desktop, plain `git` in a terminal works equally well, and every step in this guide shows both. Because CAD files cannot be merged, the one coordination rule is to say in the team chat which files you have open. Budget 45 min for the first push and 30 min for the CAD setup.

AUTHOR(S) Daiwei Lu `daiwei.lu@vanderbilt.edu`

KEYWORDS onboarding, git, github, github desktop, push, pull, CAD, member introduction

REVIEW BY 2027-09-01

Document Control

DOCUMENT ID	GEN-HG-001	REVISION	**D**
SUBSYSTEM	General / Onboarding	STATUS	Released
DOC TYPE	How-To Guide	LAST UPDATED	2026-08-17
SEASON	2027	REVIEW BY	2027-09-01
AUTHOR(S)	Daiwei Lu daiwei.lu@vanderbilt.edu	SUPERSEDES	–

Review & Approval

Name	Role	Signature	Date
Reviewing lead	Lead		

Revision History

Rev	Date	Author	Summary of Changes
A	2025-08-20	D. Lu	First release for the 2026 intake.
B	2026-08-13	D. Lu	Rewritten around the workflow we actually run, GitHub Desktop or plain push/pull on `main`. Git LFS moved to a future-work section, and file naming flagged as not yet finalised.
C	2026-08-14	D. Lu	Added the member introduction push as the first exercise, and retitled to cover both repositories.
D	2026-08-17	D. Lu	Editorial pass for neutral headings and a more consistent register throughout.

Master copy: `https://github.com/vanderbilt-motorsports/documentation`

Contents

Document Control 2
1 Overview 4
2 First Exercise: the Introductions Repository 4
3 One-Time Setup for the CAD Repository 6
4 The Working Cycle 6
5 Branches 7
6 Recovering from Mistakes 8
7 File Naming (Not Finalised Yet) 9
8 Repository Locations 9
9 Points of Contact 9
10 Quick Reference 9
11 Future Work: Git LFS and File Locking 9
Open Items 11

1 Overview

CAD lives in a GitHub repository, alongside the team's software. Prior experience with git is not required:

- **Most of the team uses GitHub Desktop**, a point-and-click application that covers the four operations we need. It fetches the latest files, shows what changed, records a snapshot, and uploads it.
- **CS/ECE members**, and anyone else comfortable in a terminal, can use plain `git` commands instead. Every step in this guide shows both forms.

CAD files are binary, so two people's changes to the *same file* cannot be merged, and one of the two versions has to be discarded. This limitation shapes the whole workflow. Until file locking is adopted (see Section 11), the team relies on the convention below.

Key Takeaway

Pull before starting work, push when you stop, and say in the team chat which files you have open. Until file locking is adopted, that message serves as the lock.

Before You Start

- ☐ You are on the team roster and have a GitHub account
- ☐ A lead has added you to the GitHub organisation
- ☐ You have written your member introduction in Overleaf (`GEN-HG-002`) and downloaded both the PDF and the `.tex`
- ☐ You have read this document to the end *before* cloning anything

2 First Exercise: the Introductions Repository

Complete this exercise before setting up the CAD repository. It uses the same workflow as everything else, on a repository where mistakes have no consequences for the car, which makes it the appropriate place to learn. You will need the two files produced in `GEN-HG-002`, meaning your introduction as a PDF and as a `.tex`.

Step 1. Clone the introductions repository. GitHub Desktop: **File → Clone Repository**, pick `VanderbiltMotorsports/MemberIntroductions`.

```
git clone git@github.com:VanderbiltMotorsports/MemberIntroductions.git
```

If the repository is not visible, the organisation invitation has usually not been accepted yet. Check your email, then ask a lead.

Step 2. Open the cloned folder on your computer. Inside, find the folder for the year you joined, such as `2026/`, or create it if you are the first one in this year.

Step 3. Copy both of your files into it, named `surname-firstname.tex` and `surname-firstname.pdf`, lowercase, no spaces:

```
2026/lu-daiwei.tex
2026/lu-daiwei.pdf
```

Surname first, so that the folder sorts alphabetically by family name.

Step 4. Back in GitHub Desktop, your two files appear in the **Changes** tab. Write a summary such as `add Daiwei Lu introduction` and click **Commit to main**.

```
git add 2026/lu-daiwei.tex 2026/lu-daiwei.pdf
git commit -m "add Daiwei Lu introduction"
```

Step 5. Click **Push origin**.

```
git push
```

Step 6. Open the repository on github.com and confirm your files are there. Tell a lead you are done.

Key Takeaway

This exercise contains the entire workflow of cloning once and then pulling, changing, committing, and pushing. Every other repository on this team, including CAD, documentation, and test data, works the same way, and the rest of this guide applies the same cycle to the CAD repository.

Reason for pushing the `.tex` as well as the PDF

The PDF is the readable copy. The `.tex` is plain text, so when an introduction is updated in a later year, GitHub can show a readable diff of exactly what changed, which it cannot do for a PDF. The whole archive is written in LaTeX for the same reason.
To edit it later, drop your `.tex` back into a copy of the Overleaf template project as `main.tex`, change what you want, and push the new pair of files.

Re-read the file before committing

The repository is private, but everyone in the organisation can read it, including members who have not joined yet, and git history is effectively permanent, and deleting a file in a later commit does not remove it from the history. Check that you have not included a home address, phone number, student ID, date of birth, grades, class schedule, or anything about somebody else. If something sensitive is pushed by mistake, do not simply delete it in a new commit. Tell a lead, because removing content from git history is a job for someone who has done it before.

3 One-Time Setup for the CAD Repository

Step 1. Ask a lead to invite you to the GitHub organisation, since the invitation has to come from them.

Allow up to a day for the invitation to arrive.

Step 2. Install **GitHub Desktop** from `https://desktop.github.com` and sign in with your GitHub account when it asks.

Terminal users install git instead and set up an SSH key under GitHub → Settings → SSH keys, because GitHub no longer accepts passwords over HTTPS.

Step 3. Get the repository onto your machine. In GitHub Desktop, choose **File → Clone Repository**, pick `vanderbilt-motorsports/vum27-cad`, choose where it lives, and note that location.

```
# terminal equivalent
git clone git@github.com:vanderbilt-motorsports/vum27-cad.git
```

Step 4. Install the CAD package using the student licence. The licence key lives in the pinned message in the team chat, never in this document or in the repository.

Step 5. Open the CAD package and point its project folder at the cloned folder from step 3.

If this step is skipped, every external reference in the assembly appears broken, which is easy to mistake for a corrupted repository.

4 The Working Cycle

Each working session follows the same four steps used in Section 2, with one added habit that matters because CAD files cannot be merged. Carry them out in this order.

Step 1. **Pull first, always.** In GitHub Desktop, click **Fetch origin**, then **Pull origin** if it offers. Working on a stale copy usually means redoing the work later.

```
git pull
```

Step 2. **Say what you are opening** in the team chat before you open it, for example “working on the front upright until tonight”. If someone has already claimed that file, wait or coordinate with them.

Step 3. **Work and commit.** Save in CAD as normal. When you reach a point worth keeping, GitHub Desktop shows the changed files in the **Changes** tab. Write a one-line summary of what you did and click **Commit to main**.

```
git add cad/suspension/front-upright.sldprt
```

```
git commit -m "front upright: pocket the inboard face, -38 g"
```

Commit messages form the logbook of the car, so state what changed and why in one line. Summaries like "changes" carry no information.

Step 4. **Push when you stop** (**Push origin** in GitHub Desktop) and say in the chat that you are out of the file.

```
git push
```

A commit that has not been pushed exists only on your laptop and is lost if the laptop fails.

Conflicts on the same file

If two people do edit the same file, git reports a **conflict** when the second person pulls or pushes. A CAD file cannot be merged, so talk to the other person, decide together whose version survives, and the other redoes their change on top of it. GitHub Desktop's conflict dialog lets you choose which copy to keep. Resolving a conflict typically costs an evening of work, which is the reason for the chat call-out in the working cycle.

5 Branches

The default is to commit directly to `main`, because the team is small enough that the chat call-out provides sufficient coordination. A **branch**, a parallel copy of the repository that can be worked on without touching `main`, is worth the extra step in three cases:

- the change is **risky** and may need to be abandoned;
- the work will take **several days** and unfinished progress should be pushed without landing on everyone;
- a **reviewed record** is wanted. In that case, push the branch, open a pull request on GitHub with a screenshot and one sentence, and ask a lead to review it before it merges.

In GitHub Desktop, choose **Current Branch** → **New Branch**, work as normal, and use **Publish branch** when you push.

```
git switch -c front-upright-pocketing
git push -u origin front-upright-pocketing
```

6 Recovering from Mistakes

Mistakes in git are almost always recoverable. When something breaks, work through these steps in order as soon as you notice, and the problem stays small.

Step 1. **Stop working.** Attempting further changes to fix the problem is what turns a small one into an unrecoverable one.

Step 2. **Do not force-push and do not delete anything.** Git almost certainly still has your work.

Step 3. **If you have not committed yet**, throw away the local change: GitHub Desktop → right-click the file in **Changes** → **Discard changes**.

```
git restore cad/suspension/front-upright.sldprt
```

Step 4. **If you committed but have not pushed**, undo the commit and keep the file as it is on disk: GitHub Desktop → **Undo** at the bottom of the commit box.

```
git reset --soft HEAD~1
```

Step 5. **If a bad commit reached** `main`, do not rewrite history. Revert it instead, which adds a new commit undoing the old one, via **History** → right-click the commit → **Revert changes in commit**.

```
git revert <commit-hash>
```

Step 6. **Say something immediately** in the team chat. Reporting a problem early is always acceptable, while concealing one delays the fix and compounds the damage.

Key Takeaway

Report breakage as soon as you notice it. Cleanup is quick while the mistake is recent and becomes slower once others have pulled on top of it.

Backups

GitHub is the backup, and deliberately the only one, with no separate nightly copy. Unpushed work is not backed up anywhere, so push at the end of every working session even if the work is unfinished. A commit marked “WIP, mounting bosses roughed in” is preferable to losing the file.

7 File Naming (Not Finalised Yet)

The CAD file naming scheme has **not** been settled yet. Until it is, follow these interim rules, chosen so that whatever scheme we adopt later can be applied in a single renaming pass:

- **Descriptive, lowercase, hyphens, no spaces:** `front-upright.sldprt`, not `Front Upright FINAL v2 (Sams copy).sldprt`.
- **No version numbers in filenames.** Git records the version history, so suffixes like `-v2-final` duplicate it and fall out of date.
- **Keep files in their area folder:** `cad/suspension/`, `cad/brakes/`, and so on.
- **Do not rename or move a file others reference** without saying so in the chat first. Renames break assembly references, and the breakage surfaces later on someone else's machine.

The leading candidate is to mirror the document ID scheme (`SUB-TT-NNN`, so a part might be `SUS-PT-014`), which would make parts and the documents about them findable by the same name. It has *not* been adopted, so do not start numbering files on your own.

[OPEN ITEM] *Finalise the CAD file naming scheme and rename the existing tree to match, in one commit (D. Lu, 2026-12-01)*

8 Repository Locations

Table 1. Team repositories and paths

What	Where
CAD repository	`github.com/vanderbilt-motorsports/vum27-cad`
Master assembly	`cad/vum27-top-level.sldasm`
Area folders	`cad/suspension/`, `cad/brakes/`, ...
Documentation repo	`github.com/vanderbilt-motorsports/documentation`
Test data	`github.com/vanderbilt-motorsports/testdata`
Released PDFs	Releases tab of the documentation repo

9 Points of Contact

10 Quick Reference

11 Future Work: Git LFS and File Locking

Two limitations of the current setup will grow with the repository, and the planned fix for both is **Git LFS** (Large File Storage). We are **not** using it yet, and nothing in this section requires action. It is recorded here so the reasoning is available when the limits are reached.

Table 2. Escalation path

Question	Ask
“How does this part work?”	Whoever owns that area (a lead knows who)
“I broke something in git”	The team chat, immediately
“I need repo access”	Any lead
“Someone is in the file I need”	That person, then a lead
“Is this design decision OK?”	Bring it to the weekly design review

Table 3. Common operations in both interfaces

What you want	GitHub Desktop	Terminal
Get the latest work	Fetch origin, then Pull	`git pull`
See what you changed	Changes tab	`git status`
Record a snapshot	Summary + Commit to main	`git add` + `git commit`
Send it to GitHub	Push origin	`git push`
Discard uncommitted work	Right-click → Discard	`git restore <file>`
Undo an unpushed commit	Undo (below commit box)	`git reset --soft HEAD~1`
Undo a pushed commit	History → Revert	`git revert <hash>`
Start a branch	Current Branch → New	`git switch -c <name>`

- **Repository size.** Plain git keeps every version of every binary forever, so a 40 MB assembly saved fifty times costs two gigabytes to clone. LFS stores large binaries outside the history and fetches only the current version, which keeps clones fast.
- **File locking.** LFS can mark CAD files lockable, so that `git lfs lock` claims a file and everyone else sees it as read-only until `git lfs unlock`. This would replace the chat-message convention from Section 4 with an enforced mechanism.

Adoption is a one-time job of about half a day. It involves installing LFS support, listing the covered extensions in a `.gitattributes` file (`*.sldprt`, `*.sldasm`, and so on), migrating the existing files, and having everyone run `git lfs install` once on their machine. GitHub Desktop supports LFS transparently, so the working cycle in this guide would not change, and locking would be added to it.

A known LFS pitfall, for whoever does the migration

If someone clones before running `git lfs install`, every CAD file arrives as a placeholder of a few hundred bytes that will not open, which closely resembles a corrupted repository. The fix is `git lfs pull`. Re-cloning does not help, and the placeholders must not be committed back.

[OPEN ITEM] *Evaluate and adopt Git LFS with file locking before the repo outgrows plain git, and run the migration in one sitting (D. Lu, 2027-03-01)*

Keeping this guide current

This guide goes stale whenever a repository name or a tool changes. If something here no longer matches reality, fix it and add a revision line at the top rather than leaving it for someone else. The archive exists so that the next person does not have to work it out from scratch.

[OPEN ITEM] *Add screenshots of GitHub Desktop and the assembly tree once the VUM-27 top level is frozen (D. Lu, 2027-01-10)*

Open Items

Owner	Item	Target Date
D. Lu	Finalise the CAD file naming scheme and rename the existing tree to match, in one commit	2026-12-01
D. Lu	Evaluate and adopt Git LFS with file locking before the repo outgrows plain git, and run the migration in one sitting	2027-03-01
D. Lu	Add screenshots of GitHub Desktop and the assembly tree once the VUM-27 top level is frozen	2027-01-10